DEFINITIVE CURRICULUM | CONCEPT-BY-CONCEPT SYLLABUS | FIRST PRINCIPLES TO AGI SYSTEMS

CUSTOMIZED FOR VIJAY SEKAR

The Complete AI Creator & Artificial Intelligence Architecture Syllabus

A concept-by-concept textbook blueprint: From Python memory and linear algebra to Transformer attention, autonomous cognitive brains, and production AI products.

CANDIDATE	STRUCTURE	SCOPE	ULTIMATE GOAL
Vijay Sekar	**10 Chapters • 60 Concepts**	**Math + Code + Brain Arch**	**AI Creator & Product Founder**

🧭 How to Learn From This Syllabus

Vijay, this is your granular master index. Every chapter below is organized into exact sequential micro-concepts. Do not skip steps. For each concept, study the **Core Intuition**, master the exact **Syntax/Math**, complete the **Immediate Practice Task**, and observe how it connects directly into the next topic. By Chapter 10, you will have built an artificial mind from the ground up.

📘 Chapter 1: Python for AI & Modern Systems Engineering

CONCEPT 1.1 Environment Management & Package Isolation (uv)

The Core Intuition: AI relies on heavy, conflicting binary packages (PyTorch, CUDA, NumPy). Installing them globally corrupts your computer. uv is an ultra-fast Rust-based Python package manager.

Why It Matters for AI: You must be able to spin up fresh environments with exact C++ and CUDA bindings in seconds.

What to Master: `uv venv`, `uv pip install torch` with CUDA wheel index, and locking dependencies in `pyproject.toml`.

> **Immediate Practice Task:** Create a project folder, initialize a Python 3.12 venv with uv, install `httpx` and `pydantic`, and verify your active environment in PowerShell.

CONCEPT 1.2 Core Data Structures & Memory Under the Hood

The Core Intuition: Python lists, dictionaries, tuples, and sets have fundamentally different memory layouts and Big-O time complexities (O(1) vs O(n)).

Why It Matters for AI: Datasets contain millions of tokens. Searching an O(n) list inside a training loop instead of an O(1) hash map makes your code 1,000x slower.

What to Master: Hash table collisions, key hashing, list over-allocation, and the `collections` module (`Counter` , `defaultdict` , `deque` for fixed-size context buffers).

> **Immediate Practice Task:** Write a script that loads a 100,000-word text file, tokenizes into words, and uses `collections.Counter` to output the top 50 tokens in under 100ms.

CONCEPT 1.3 Generators, Iterators & Memory-Efficient Streaming

The Core Intuition: A regular function returns a list all at once. A generator uses `yield` to produce items one at a time on-demand, consuming virtually zero RAM.

Why It Matters for AI: Multi-gigabyte training files cannot fit into memory. Every PyTorch DataLoader and streaming LLM response operates as an iterator.

What to Master: The `yield` keyword, generator expressions, and `itertools.batched` for assembling mini-batches.

> **Immediate Practice Task:** Write a generator function `stream_jsonl(path)` that streams a simulated 5GB file line-by-line while keeping RAM usage under 20MB.

CONCEPT 1.4 Object-Oriented Programming for AI Model Layers

The Core Intuition: Neural network layers hold weights (state) and perform math (behavior). OOP encapsulates this neatly.

Why It Matters for AI: Every deep learning library (PyTorch, Keras) is built on classes where module instances are callable like functions via `__call__`.

What to Master: Classes, inheritance, the `__call__` dunder method, and `@dataclass` for clean configuration objects.

> **Immediate Practice Task:** Build a custom class `LinearLayer` with random weights that implements `__call__(self, x)` to execute y = xW + b.

CONCEPT 1.5 Modern Type Hinting & Pydantic v2

The Core Intuition: Pydantic enforces strict runtime types and automatically parses untrusted incoming data into validated objects.

Why It Matters for AI: LLMs return messy text. You must force the model to output valid JSON matching an exact schema so your software never crashes.

What to Master: `pydantic.BaseModel`, `Field(description=...)`, type unions (`|`), and custom validators.

> **Immediate Practice Task:** Build a Pydantic schema for an Invoice with line items, tax, and total. Safely parse and validate a raw JSON string from an LLM.

CONCEPT 1.6 Asynchronous Python (`asyncio` & `httpx`)

The Core Intuition: Synchronous code freezes while waiting for network responses. Async pauses execution (`await`), allowing hundreds of concurrent calls.

Why It Matters for AI: AI model API calls have 200ms–5000ms latency. Sync code bottlenecks your app to 1 request at a time.

What to Master: `async def`, `await`, `asyncio.gather()`, `httpx.AsyncClient`, and rate limiting with `asyncio.Semaphore`.

> **Immediate Practice Task:** Write an async script using `httpx` making 20 concurrent simulated LLM API calls bounded by a semaphore of 5.

CONCEPT 1.7 Serialization & File Formats (JSON, JSONL, Parquet)

The Core Intuition: Converting Python objects into bytes on disk and back rapidly.

Why It Matters for AI: JSONL is the universal format for fine-tuning datasets; Parquet stores millions of vector embeddings with 10x compression.

What to Master: Fast reading/writing of `.jsonl` files and columnar Parquet files using `pyarrow`.

> **Immediate Practice Task:** Write a utility script reading a raw text corpus, formatting it into ChatML pairs, and streaming to a compressed JSONL file.

🎯 Milestone Project 1: Asynchronous Dataset Ingestion & Validation Engine

A high-throughput CLI tool that ingests messy input documents, runs asynchronous validation pipelines via Pydantic and asyncio, sanitizes invalid records, and outputs a pristine, token-budgeted `dataset.jsonl` file ready for model training.

Acceptance Criteria: Processes 10,000 records in <5 seconds • Uses generators for zero RAM overflow • Rejects 100% of malformed rows with error logs.

📐 Chapter 2: The Computational Mathematics of Intelligence

CONCEPT 2.1 The Geometric Intuition of AI (Vectors & High-Dimensional Space)

The Core Intuition: Computers only understand numbers. Every word, sentence, or image is converted into a list of numbers called a Vector. Semantic meaning is mapped to geometric position.

Why It Matters for AI: Words with similar meanings point in nearly the same direction in 1,536-dimensional space.

What to Master: Dot Product (a · b = sum(a_i * b_i)), Vector Norms (L2), and Cosine Similarity: cos(theta) = (a · b) / (||a|| * ||b||).

> **Immediate Practice Task:** Compute the dot product and cosine similarity between [1, 2, 3] and [2, 4, 6] in Python. Observe why the cosine similarity is exactly 1.0.

CONCEPT 2.2 NumPy Fundamentals & Memory Contiguity

The Core Intuition: A NumPy `ndarray` stores raw numbers in contiguous memory blocks, allowing CPU cache and SIMD vector hardware to process them millions of times faster than Python lists.

Why It Matters for AI: NumPy is the foundational bedrock under PyTorch, TensorFlow, and all vector search engines.

What to Master: `np.array`, `float32` vs `bfloat16`, memory layouts, and array attributes (`shape`, `dtype`, `strides`).

> **Immediate Practice Task:** Create a 1000 x 1536 random float32 NumPy array (simulating 1,000 document embeddings). Print its memory footprint in MB.

CONCEPT 2.3 Array Manipulation, Reshaping & Slicing

The Core Intuition: Data in neural networks changes shape continuously (e.g. from 2D token sequences to 4D multi-head attention matrices).

Why It Matters for AI: Shape mismatch errors are the #1 source of deep learning bugs. Mastering reshaping makes you immune to them.

What to Master: `np.reshape()`, `np.transpose()`, `np.expand_dims()`, and multidimensional slicing.

> **Immediate Practice Task:** Create a 1D array of 24 numbers. Reshape to (2, 3, 4), transpose to (2, 4, 3), and add a batch dimension to make it (1, 2, 4, 3).

CONCEPT 2.4 Vectorization & Broadcasting Rules

The Core Intuition: Broadcasting allows arithmetic operations across arrays of different shapes without copying data in memory.

Why It Matters for AI: Adding a bias vector of size 10 to a batch of 64 activations stretches the bias across the batch at hardware speed.

What to Master: The 2 rules of broadcasting and replacing slow Python `for` loops with vectorized expressions.

> **Immediate Practice Task:** Benchmark adding a bias vector of 512 numbers to 10,000 activation vectors using a Python loop vs NumPy broadcasting. Compare execution times.

CONCEPT 2.5 Matrix Multiplication & Linear Projections

The Core Intuition: Matrix multiplication is a geometric transformation that rotates, scales, and projects vectors into new conceptual spaces.

Why It Matters for AI: Every layer in an LLM and every attention head is a matrix multiplication (Y = XW). When an AI thinks, it performs billions of matrix multiplications per second.

What to Master: Matrix dimension matching rules (m x k times k x n = m x n), and the Python `@` operator.

> **Immediate Practice Task:** Multiply an input matrix of shape (32, 128) by a weight matrix of shape (128, 64) using `@`. Verify the output shape.

CONCEPT 2.6 Calculus of Learning: Derivatives, Gradients & The Chain Rule

The Core Intuition: A derivative tells you how to nudge a weight to reduce error. Backpropagation is simply applying the chain rule backward through layers.

Why It Matters for AI: This is how models learn from mistakes. Without the chain rule, neural networks could not be trained.

What to Master: Partial derivatives, the gradient vector, the chain rule (dz/dx = dz/dy * dy/dx), and Gradient Descent: W_new = W - eta * grad(L).

> **Immediate Practice Task:** Write a 15-line gradient descent loop in Python finding the minimum of f(x) = (x - 7)^2. Watch x converge to 7.

CONCEPT 2.7 Probability & Loss Math: Logits, Softmax & Cross-Entropy

The Core Intuition: Neural nets output raw scores (logits). Softmax squashes them into probabilities summing to 1.0. Cross-Entropy measures how surprised the model was.

Why It Matters for AI: All modern LLMs output vocabulary logits, apply Softmax with a Temperature parameter, and train on Cross-Entropy Loss.

What to Master: Softmax formula, the mathematical effect of Temperature (T), and Cross-Entropy Loss: -log(P_target).

> **Immediate Practice Task:** Implement `softmax(logits, temperature)` in NumPy. Test with T = 0.5, 1.0, and 2.0. Verify probabilities sum to 1.0.

🎯 Milestone Project 2: Pure NumPy Vector Search Engine

Build an in-memory vector database from scratch using only NumPy (no third-party AI libraries). Index 10,000 document vectors, receive a query vector, broadcast cosine similarity math, and return top-5 nearest neighbors in <10 milliseconds.

Acceptance Criteria: Implements true vector normalization and matrix multiplication • Zero Python loops during search • Matches formal mathematical rankings.

🧠 Chapter 3: Deep Learning From First Principles (How Neurons Learn)

CONCEPT 3.1 The Artificial Neuron (z = Wx + b)

The Core Intuition: A mathematical equation taking inputs, multiplying by weights (memory/importance), adding a bias (threshold), and firing an output.

Why It Matters for AI: Billions of these equations wired together form GPT-4, Claude, and biological brains.

What to Master: Weighted sums, bias offsets, and vector dot product representation of a neuron.

> **Immediate Practice Task:** Code a single neuron function in Python that takes 3 inputs, computes weighted sum with bias, and outputs the result.

CONCEPT 3.2 Activation Functions (ReLU, GELU, Sigmoid, Softmax)

The Core Intuition: Non-linear mathematical curves that allow networks to learn complex, non-linear real-world relationships.

Why It Matters for AI: Without non-linear activations, stacking 100 neural layers mathematically collapses into a single straight line.

What to Master: `ReLU` (max(0, z)) and `GELU` (the standard activation function inside modern Transformer models).

> **Immediate Practice Task:** Implement `relu(x)` and `gelu(x)` in Python. Compare their outputs for inputs between -3.0 and +3.0.

CONCEPT 3.3 Multi-Layer Perceptrons (MLPs) & Representation Learning

The Core Intuition: Stacking an input layer, hidden layers, and an output layer. Hidden layers learn internal representations of the world.

Why It Matters for AI: Every Transformer block contains a 2-layer MLP (Feed-Forward Network) that stores factual knowledge.

What to Master: Hidden dimensions, expanding and compressing dimensions, and multi-layer matrix transformations.

> **Immediate Practice Task:** Build a 2-layer MLP in pure Python with initialized random weights mapping an input of dimension 4 to hidden dimension 8 to output 2.

CONCEPT 3.4 Backpropagation & The Computational Graph

The Core Intuition: Building a graph of operations during the forward pass, calculating loss, and recursively propagating gradients backward to all weights via the chain rule.

Why It Matters for AI: The algorithmic breakthrough that made training deep neural networks computationally practical.

What to Master: Computational graphs, cached activations, and calculating dL/dW and dL/dX.

> **Immediate Practice Task:** Code a minimal scalar automatic differentiation class `Value` with `.backward()` for addition and multiplication (Karpathy micrograd style).

CONCEPT 3.5 Optimization Algorithms (SGD, Momentum, Adam, AdamW)

The Core Intuition: Adaptive optimizers maintain running averages of past gradients (momentum) and adjust learning rates per individual parameter.

Why It Matters for AI: 100% of modern Large Language Models are trained using **AdamW**.

What to Master: Adam's 1st moment (momentum) and 2nd moment (variance), decoupled weight decay, and learning rate warmup/decay schedules.

> **Immediate Practice Task:** Code the parameter update step of AdamW in Python: maintain m_t and v_t, apply bias correction, and update parameters.

CONCEPT 3.6 PyTorch Tensor Foundations & GPU CUDA Memory

The Core Intuition: PyTorch is NumPy on GPU steroids with automatic differentiation built-in.

Why It Matters for AI: The global industry-standard framework used to train and run frontier models.

What to Master: `torch.tensor`, device management (`.to("cuda")`), and GPU VRAM lifecycle.

> **Immediate Practice Task:** Write a script allocating two 5000 x 5000 tensors on GPU, multiplying them, and benchmarking against CPU.

CONCEPT 3.7 PyTorch Autograd & Building Custom `nn.Module` Networks

The Core Intuition: Subclassing `nn.Module` to build trainable neural networks with automated gradient calculations.

Why It Matters for AI: The exact architectural structure used to write Transformers and custom AI models.

What to Master: The 5-step PyTorch training loop: `zero_grad()`, forward, loss calculation, `loss.backward()`, `optimizer.step()`.

> **Immediate Practice Task:** Build a 3-layer neural network in PyTorch classifying synthetic moon data over 100 epochs until loss converges.

🎯 Milestone Project 3: A Neural Network Trained From Scratch in PyTorch

Build and train a complete multi-layer neural network from scratch in PyTorch to classify handwritten digits (MNIST) or tabular records, including learning rate schedulers, evaluation accuracy curves, and model state checkpoint saving/loading.

Acceptance Criteria: >95% accuracy on test set • Clean modular PyTorch nn.Module • Checkpoint saving and reloadable inference pipeline.

Chapter 4: The Transformer Architecture (The Engine of Modern AI)

CONCEPT 4.1 Tokenization Algorithms (Byte-Pair Encoding & Tiktoken)

The Core Intuition: Language models do not read characters or words. They read numeric chunks called Tokens merged iteratively by frequency.

Why It Matters for AI: Tokenization explains why LLMs struggle with character spelling, arithmetic without spaces, and why multilingual prompts cost more.

What to Master: The BPE merge algorithm, vocabulary sizes (32k–128k), special tokens, and using `tiktoken`.

> **Immediate Practice Task:** Write a toy BPE tokenizer from scratch that takes a text paragraph, finds the most frequent character pair, and merges them over 10 steps.

CONCEPT 4.2 Embedding Layers & Rotary Positional Encodings (RoPE)

The Core Intuition: Embedding layers map token IDs to dense vectors. Positional Encodings inject token order into attention.

Why It Matters for AI: Modern models (Llama 3, DeepSeek) use Rotary Position Embeddings (RoPE), rotating vectors in complex space to handle 128k+ token context windows.

What to Master: `nn.Embedding` and the mathematical rotation equations of RoPE.

> **Immediate Practice Task:** Initialize a PyTorch `nn.Embedding(1000, 64)`. Pass a batch of token IDs (2, 5) and inspect output shape (2, 5, 64).

CONCEPT 4.3 Scaled Dot-Product Self-Attention (Q, K, V Matrices)

The Core Intuition: Queries (Q) ask for context, Keys (K) announce identity, and Values (V) provide content. Attention allows every word to communicate with every other word.

Why It Matters for AI: The foundational formula powering all modern Generative AI: Attention(Q, K, V) = Softmax(QK^T / sqrt(d_k)) * V.

What to Master: Why we divide by sqrt(d_k) (to prevent vanishing gradients) and implementing scaled dot-product attention in PyTorch.

> **Immediate Practice Task:** Implement `scaled_dot_product_attention(Q, K, V)` in PyTorch using pure matrix operations. Verify output shape matches V.

CONCEPT 4.4 Multi-Head Attention & Grouped-Query Attention (GQA)

The Core Intuition: Multi-Head Attention runs parallel heads capturing different relationships. GQA shares Key/Value heads across multiple Query heads to save GPU memory.

Why It Matters for AI: GQA is the architectural standard in Llama 3 and Mistral that makes KV-caching fit in VRAM during inference.

What to Master: Head splitting, projection layers (W_Q, W_K, W_V, W_O), and MHA vs MQA vs GQA.

> **Immediate Practice Task:** Code a PyTorch `MultiHeadAttention` module that splits dimensions across 4 heads, computes attention, and projects back.

CONCEPT 4.5 Residual Connections & Pre-Layer Normalization (RMSNorm)

The Core Intuition: Residual connections add layer inputs directly to layer outputs (x + F(x)), creating an information highway that prevents gradients from vanishing.

Why It Matters for AI: Modern LLMs place RMSNorm before attention and MLP layers (Pre-LN) for stable training across 100+ layers.

What to Master: The Pre-LN Transformer block architecture and implementing RMSNorm with learnable scaling gamma.

> **Immediate Practice Task:** Implement the `RMSNorm` layer in PyTorch and verify output vectors have unit root-mean-square norm.

CONCEPT 4.6 Causal Attention Masking & Autoregressive Generation

The Core Intuition: Language models must not look into the future during training. We apply an upper-triangular mask of -infinity so token 3 only attends to tokens 1, 2, and 3.

Why It Matters for AI: Enables autoregressive next-token prediction and allows parallel training across entire sentences simultaneously.

What to Master: Causal lower-triangular masks with `torch.tril()` and autoregressive generation loops.

> **Immediate Practice Task:** Create a 4 x 4 attention matrix in PyTorch. Apply a causal mask of -infinity to upper entries and verify Softmax outputs are 0.0.

🎯 Milestone Project 4: Build a Mini-GPT Language Model From Scratch in PyTorch

Follow Andrej Karpathy's architecture to build a complete autoregressive GPT model from scratch in a single ~250-line PyTorch script: Token Embeddings + Positional Embeddings + Stacked Transformer Blocks (RMSNorm + Causal Multi-Head Attention + MLP) + Output Head. Train it on a text corpus until it generates coherent English sentences.

Acceptance Criteria: Implements pure self-attention without black-box libraries • Loss converges smoothly • Generates text token-by-token from a prompt.

🚀 Chapter 5: Pre-Training, Post-Training & Creating Frontier Reasoning

CONCEPT 5.1 The 3 Stages of Modern LLMs (Pre-training → SFT → Alignment)

The Core Intuition: Base models are raw autocomplete engines. They become assistants through Supervised Fine-Tuning (SFT) and Preference Alignment (DPO/RLHF).

Why It Matters for AI: Understanding where knowledge is stored (pre-training) vs where tone and style are learned (SFT) guides when to fine-tune vs when to prompt.

What to Master: Differences between Base and Instruct models, and preventing catastrophic forgetting.

> **Immediate Practice Task:** Query a raw Base model vs an Instruct model with `"What is photosynthesis?"` and analyze the structural difference in output.

CONCEPT 5.2 Instruction Tuning & Formatting (ChatML Schema)

The Core Intuition: Special role tokens indicate who is speaking: system, user, or assistant.

Why It Matters for AI: Wrong chat templates cause fine-tuned models to hallucinate role tokens or leak confidential system prompts.

What to Master: The ChatML specification and formatting datasets using Hugging Face's `tokenizer.apply_chat_template()`.

> **Immediate Practice Task:** Format 5 raw Q&A pairs into tokenized ChatML format with system prompts and attention masks using a Hugging Face tokenizer.

CONCEPT 5.3 Parameter-Efficient Fine-Tuning (LoRA Math: W + (alpha / r) * BA)

The Core Intuition: Freezing base model weights W and decomposing weight updates into two small low-rank matrices B and A.

Why It Matters for AI: Reduces trainable parameters by 99%, allowing you to fine-tune 8B models on consumer GPUs without losing performance.

What to Master: The LoRA formula, rank (r), alpha, and merging LoRA adapters back into base weights.

> **Immediate Practice Task:** Initialize a PyTorch linear layer (1024, 1024), freeze it, attach low-rank adapters A and B (r = 8), and verify gradient updates.

CONCEPT 5.4 4-Bit Quantization & QLoRA with Unsloth

The Core Intuition: Compressing base model weights into 4-bit NormalFloat (NF4), reducing an 8B model's footprint to 5.5GB VRAM.

Why It Matters for AI: Using `Unsloth` Triton kernels allows 5x faster training with 80% less memory on accessible GPUs.

What to Master: NF4 quantization, Double Quantization, and training loss tracking in Weights & Biases (WandB).

> **Immediate Practice Task:** Install `unsloth` in Colab/GPU, load `Llama-3.2-3B-Instruct-bnb-4bit`, attach LoRA adapters, and print trainable parameters.

CONCEPT 5.5 Alignment: DPO (Direct Preference Optimization) & RLHF

The Core Intuition: Showing the model pairs of chosen (good) and rejected (bad) responses, increasing chosen likelihood while decreasing rejected likelihood.

Why It Matters for AI: Replaced complex RLHF pipelines with a simple, stable loss function to align models with human values.

What to Master: DPO dataset formats and running alignment loops using Hugging Face TRL (`DPOTrainer`).

> **Immediate Practice Task:** Create a 10-example preference dataset (concise vs verbose answers) and run a toy DPO training script with TRL.

CONCEPT 5.6 Test-Time Compute & Reasoning Models (<think> Tokens & DeepSeek-R1)

The Core Intuition: Allowing the model to think before answering by generating internal chains-of-thought inside `` tags, exploring hypotheses and backtracking.

Why It Matters for AI: The new scaling frontier: smaller models outperform massive models when given test-time compute to think through problems.

What to Master: Chain-of-Thought (CoT), verifiable reward models, and DeepSeek-R1 GRPO reinforcement learning mechanics.

> **Immediate Practice Task:** Prompt DeepSeek-R1 or Qwen-Coder with a logic riddle. Parse and inspect the internal thinking trace for self-corrections.

🎯 Milestone Project 5: Fine-Tune an Open-Source 8B Model on Custom Domain Knowledge

Select a vertical domain (e.g. Legal, Medical Triage, or Manga Storyboarding). Synthesize 2,000 instruction-response pairs. Use Unsloth and QLoRA to fine-tune Llama 3.3 8B or Qwen 2.5 7B. Track loss in WandB, export LoRA adapters, merge weights, and benchmark against the base model.

Acceptance Criteria: Strict adherence to domain schema • Quantifiable accuracy improvement on domain tests • Merged weights ready for vLLM deployment.

📋 The Master Concept Checklist (Concepts 1.1 to 10.4)

- **1.1:** Environment Management (`uv`)
- **1.2:** Core Data Structures & Memory
- **1.3:** Generators & Streaming
- **1.4:** OOP for AI Model Layers
- **1.5:** Type Hinting & Pydantic v2
- **1.6:** Asynchronous Python (`asyncio`)
- **1.7:** Serialization (JSONL, Parquet)
- **2.1:** Geometric Intuition of Vectors
- **2.2:** NumPy Memory & Arrays
- **2.3:** Reshaping, Slicing & Transposing
- **2.4:** Vectorization & Broadcasting
- **2.5:** Matrix Multiplication (`@`)
- **2.6:** Calculus, Gradients & Chain Rule
- **2.7:** Probability, Softmax & Loss
- **3.1:** The Artificial Neuron (z = Wx + b)
- **3.2:** Activation Functions (ReLU, GELU)
- **3.3:** Multi-Layer Perceptrons (MLPs)
- **3.4:** Backpropagation from Scratch
- **3.5:** AdamW & Learning Rate Schedules
- **3.6:** PyTorch Tensors & CUDA VRAM
- **3.7:** PyTorch `nn.Module` Training Loop
- **4.1:** Tokenization & BPE Mechanics
- **4.2:** Embeddings & RoPE Encodings
- **4.3:** Scaled Dot-Product Self-Attention
- **4.4:** Multi-Head Attention & GQA
- **4.5:** Residuals & RMSNorm Pre-LN
- **4.6:** Causal Masking & Generation
- **5.1:** Pre-Training vs SFT vs Alignment
- **5.2:** ChatML & Instruction Formatting
- **5.3:** LoRA Mathematics (W + BA)
- **5.4:** QLoRA 4-bit with Unsloth
- **5.5:** DPO Alignment with TRL
- **5.6:** Reasoning Models & Test-Time Compute
- **6.1:** Semantic Spaces & Distance Metrics
- **6.2:** HNSW & IVFFlat Indexing
- **6.3:** Document Chunking Strategies
- **6.4:** PostgreSQL & `pgvector` Setup
- **6.5:** Hybrid Search (BM25 + Vectors)
- **6.6:** Cross-Encoder Re-Ranking
- **6.7:** Automated RAG Evals (Ragas)

7.1: Cognitive Brain Architecture

7.2: ReAct & Reflection Patterns

7.3: State Machines with LangGraph

7.4: Agent Memory Systems

7.5: Tool Sandboxing (Docker / E2B)

7.6: Multi-Agent Supervisor Teams

8.1: Vision-Language Models & ViT

8.2: Whisper Audio & Mel-Spectrograms

8.3: Real-Time Voice WebSockets

9.1: Production Serving with `vLLM`

9.2: Prompt Caching & Redis Cache

9.3: AI Security & NeMo Guardrails

9.4: Observability with Langfuse

10.1: Full-Stack AI (Next.js + FastAPI)

10.2: Streaming UI & Vercel AI SDK

10.3: Stripe Credit Metering & Margins

10.4: Serverless GPU Hosting (Modal)

PanelFlow Studio Knowledge Base • Prepared for Vijay Sekar • Complete Concept-by-Concept AI Curriculum